Université Abdelmalek Essaadi
Ecole Nationale des Sciences
Appliquées Al Hoceima

Compte rendu – TP5 JEE :

Gestion des Produits – MVC2

Application Web JEE avec Authentification et Gestion des Rôles

Réalisé par : Hajar Elyazri

Filière : TDIA2

Encadré par : Pr. Mohammed cherradi

Année universitaire : 2025/2026

1. Introduction

1.1 Objectif du projet

L'objectif de ce TP est de faire évoluer l'application web Java EE développée lors du TP4 (MVC1 avec données en mémoire) vers une architecture robuste et persistante en intégrant :

- JPA / Hibernate 3 comme couche de persistance (ORM)
- MySQL comme système de gestion de base de données relationnelle
- Maven comme outil de gestion de dépendances et de build
- Architecture MVC2 conservée (Servlets + JSP)

L'application gère les produits (CRUD complet) avec un système d'authentification et de gestion des rôles :

- **ADMIN : gestion complète (consultation, ajout, modification, suppression)**
- **USER : consultation des données uniquement (lecture seule)**

1.2 Technologies utilisées

Technologie	Rôle
Java EE (Servlets)	Contrôleurs MVC2 — traitement des requêtes HTTP
JSP + JSTL	Vues — affichage dynamique des données
JPA 2.0 / Hibernate 3.6	ORM — persistance des entités en base de données
MySQL 8	Base de données relationnelle
Maven	Gestion des dépendances et build du projet (.war)
Apache Tomcat 9	Serveur d'application pour le déploiement
HTML / Bootstrap 5	Interface utilisateur responsive
Servlet Filters	Sécurité : authentification et contrôle des rôles
Pattern DAO	Couche d'accès aux données via EntityManager
Pattern Singleton	Gestion centralisée de la couche métier

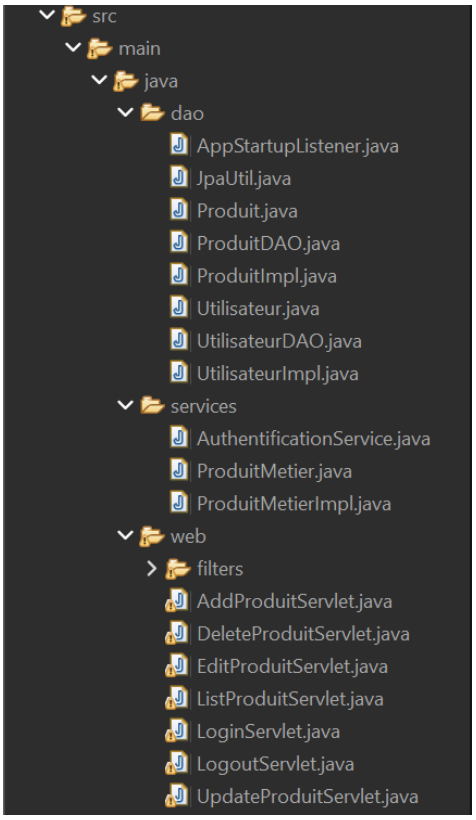
2. Architecture de l'application

2.1 Architecture MVC2 (Modèle – Vue – Contrôleur)

L'application suit le patron de conception MVC2 dans lequel les Servlets jouent le rôle de Contrôleurs, les JSP celui de Vues, et les entités JPA celui de Modèles.

Couche	Composants	Responsabilité
Modèle (Model)	Produit.java, Utilisateur.java	Entités JPA mappées en base MySQL
Vue (View)	index.jsp, login.jsp, *.jsp	Affichage HTML dynamique via JSTL
Contrôleur (Controller)	Servlets (web/)	Traitement des requêtes, appel du service
Couche Service	ProduitMetierImpl, AuthentificationService	Logique métier, orchestration
Couche DAO	ProduitImpl, UtilisateurImpl	Accès BDD via EntityManager JPA
Filtres	AuthFilter, RoleFilter	Sécurité transversale (AOP-like)

2.2 Structure des packages



2.3 Flux de traitement d'une requête

Le diagramme de séquence simplifié ci-dessous illustre le flux pour une opération CRUD (ex. : affichage de la liste des produits) :

Étape	Composant	Action
1	Navigateur	GET /listProduits
2	AuthFilter	Vérification de la session (utilisateur connecté ?)
3	ListProduitServlet	Appel de ProduitMetierImpl.getAllProduits()
4	ProduitMetierImpl	Délègue à ProduitImpl.getAllProduits()
5	ProduitImpl (JPA)	EntityManager → SELECT * FROM produit (MySQL)
6	ListProduitServlet	req.setAttribute("listeProduits", liste)
7	index.jsp	Rendu HTML avec <c:forEach> JSTL
8	Navigateur	Affichage de la page

3. Fonctionnalités implémentées

3.1 Gestion des produits (CRUD)

L'application permet la gestion complète des produits à travers les opérations CRUD, toutes persistées en base MySQL via JPA/Hibernate.

Fonctionnalité	URL	Méthode HTTP	Rôle requis
Afficher tous les produits	listProduits	GET	USER / ADMIN
Rechercher un produit par ID	listProduits?idProduit=2	GET	USER / ADMIN
Ajouter un produit	addProduit	POST	ADMIN
Préparer la modification	editProduit?id=2	GET	ADMIN
Modifier un produit	updateProduit	POST	ADMIN
Supprimer un produit	deleteProduit?id=2	GET	ADMIN

3.2 Authentification

Le système d'authentification vérifie les credentials en base MySQL via une requête JPQL, puis stocke l'utilisateur en session HTTP.

Fonctionnalité	JRL	Description
Page de connexion	login (GET)	Affichage du formulaire login / mot de passe
Traitement de connexion	login (POST)	Vérification JPQL en BDD + création de session
Déconnexion	logout	Invalidation de la session HTTP

3.3 Gestion des rôles

Deux filtres Servlet assurent la sécurité de manière transversale, sans modifier la logique des Servlets.

Filtre	URL(s) surveillée(s)	Action
AuthFilter	/* (toutes les URLs)	Redirige vers /login si pas de session active
RoleFilter	/addProduit, /editProduit, /updateProduit, /deleteProduit	Redirige vers accessDenied.jsp si rôle != ADMIN

Rôle	Droits
ADMIN	Consultation, ajout, modification et suppression des produits
USER	Consultation uniquement (lecture seule) — boutons Modifier/Supprimer masqués

3.4 Gestion des erreurs

Type d'erreur	Page affichée
404 – Page non trouvée	error404.jsp
500 – Erreur serveur	error500.jsp
403 – Accès refusé	accessDenied.jsp
Login / mot de passe incorrect	Message d'erreur inline sur login.jsp

3.5 Comptes de test

Les comptes sont insérés automatiquement en base MySQL par AppStartupListener au premier démarrage si la table est vide.

Login	Mot de passe	Rôle
admin_hajar	admin123	ADMIN
user_hajar	user123	USER

4. Intégration JPA / Hibernate 3

4.1 Configuration (persistence.xml)

Le fichier persistence.xml placé dans src/main/resources/META-INF/ configure la connexion MySQL et le comportement d'Hibernate :

Propriété Hibernate	Valeur	Description
hibernate.connection.driver_class	com.mysql.cj.jdbc.Driver	Driver JDBC MySQL
hibernate.dialect	MySQL5InnoDBDialect	Dialecte SQL généré par Hibernate
hibernate.hbm2ddl.auto	update	Crée/met à jour les tables automatiquement
hibernate.show_sql	true	Affiche les requêtes SQL dans la console

4.2 Annotations JPA sur les entités

Annotation	Cible	Rôle
@Entity	Classe	Déclare la classe comme entité JPA persistée
@Table(name=...)	Classe	Mappe vers la table MySQL correspondante
@Id	Champ id	Déclare la clé primaire
@GeneratedValue(IDENTITY)	Champ id	Auto-incrément géré par MySQL
@Column(name=...)	Champ	Mappe vers la colonne de la table

4.3 Opérations JPA dans ProduitImpl

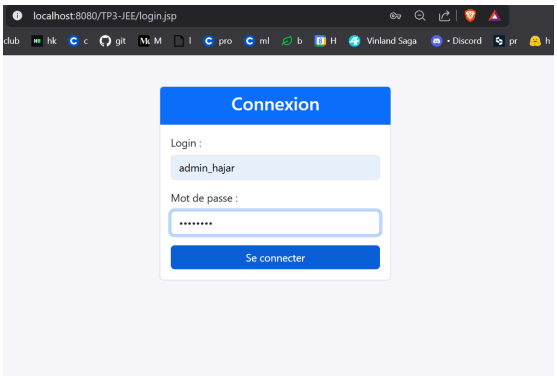
Opération CRUD	Méthode JPA utilisée	Requête générée
Ajouter	em.persist(produit)	INSERT INTO produit (nom, description, prix) VALUES (?, ?, ?)
Modifier	em.merge(produit)	UPDATE produit SET nom=?, description=?, prix=? WHERE id=?
Supprimer	em.remove(em.find(...))	DELETE FROM produit WHERE id_produit=?
Lire par ID	em.find(Produit.class, id)	SELECT * FROM produit WHERE id_produit=?
Lire tout	JPQL : SELECT p FROM Produit p	SELECT * FROM produit

5. Comparaison TP4 (MVC1) vs TP5 (JPA/Hibernate)

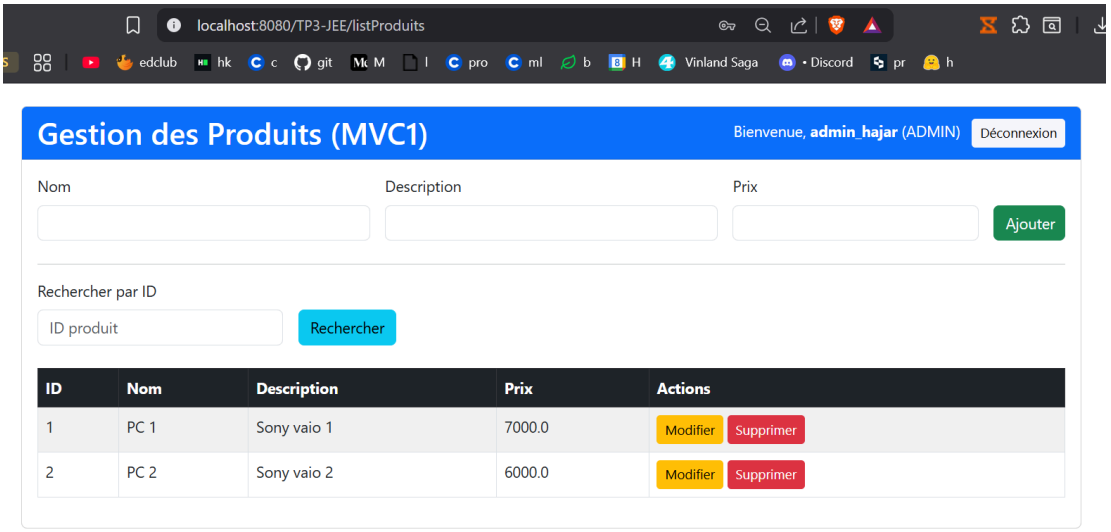
Aspect	TP4 — MVC1 (mémoire)	TP5 — JPA/Hibernate (MySQL)
Persistance	ArrayList en mémoire (volatile)	Base de données MySQL (durable)
ProduitImpl	Opérations sur List<Produit>	EntityManager JPA (persist, merge, remove, find)
UtilisateurImpl	static List<Utilisateur>	Requêtes JPQL (findByLogin, findByLoginAndPassword)
Initialisation données	Méthode init() dans ProduitImpl	AppStartupListener (@WebListener)
Gestion de build	Classpath Eclipse manuel	Maven (pom.xml) — dépendances automatiques
Perte de données	À chaque redémarrage Tomcat	Aucune — données sauvegardées en MySQL
Servlets / Filters / JSP	Identiques	Identiques (aucune modification)
web.xml	Identique	Identique (aucune modification)

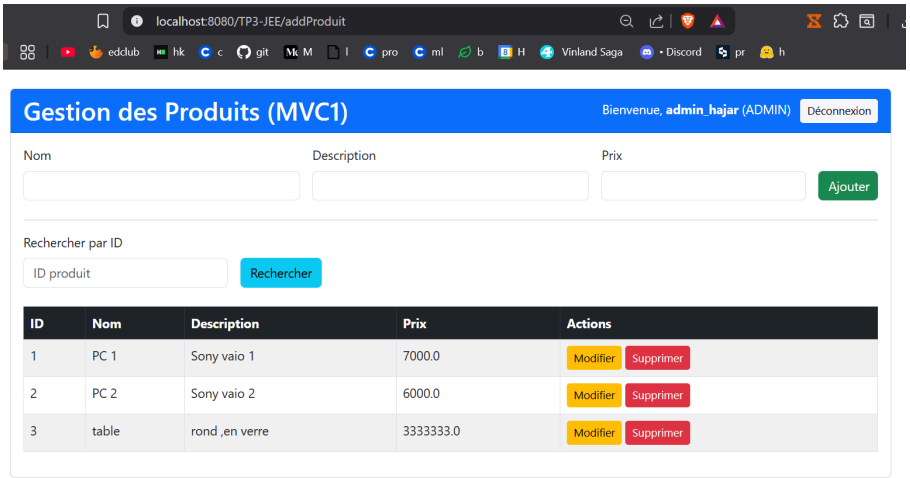
4. CAPTURES D'ÉCRAN

Page 1 : Connexion

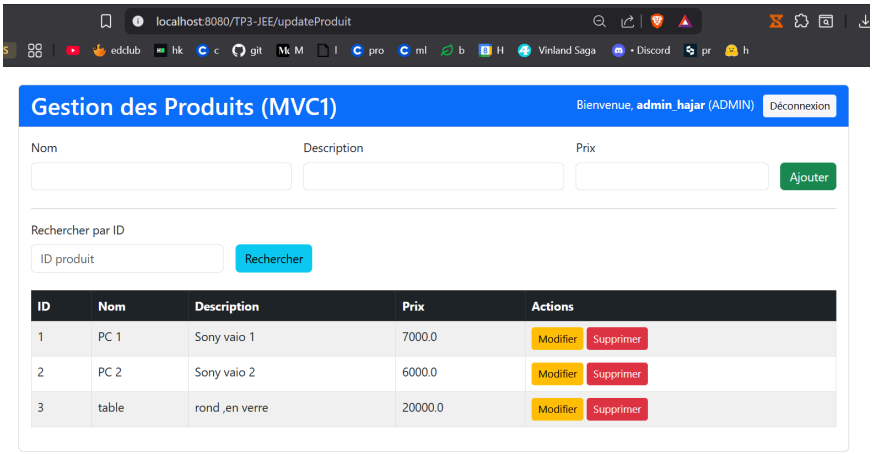


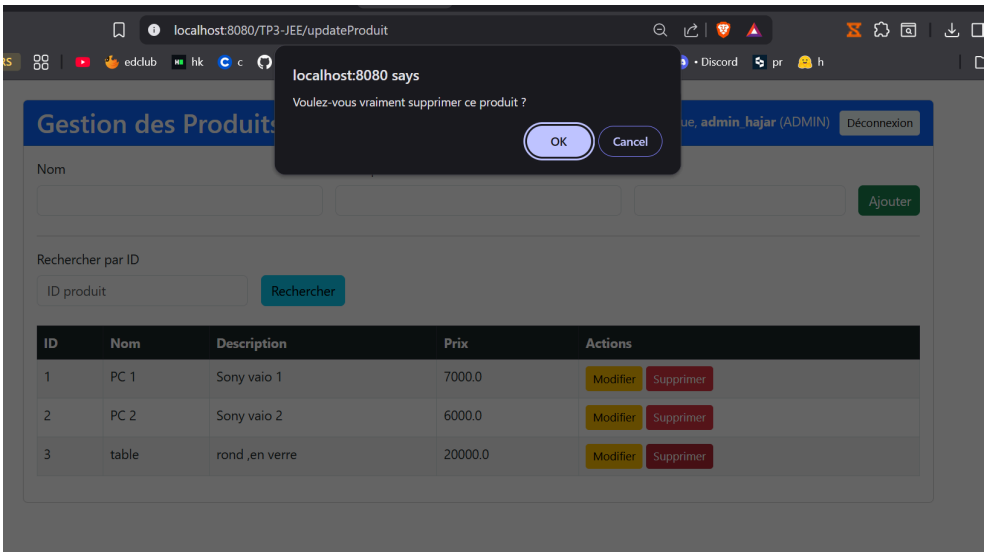
Page 2 : Liste des produits (après connexion ADMIN)

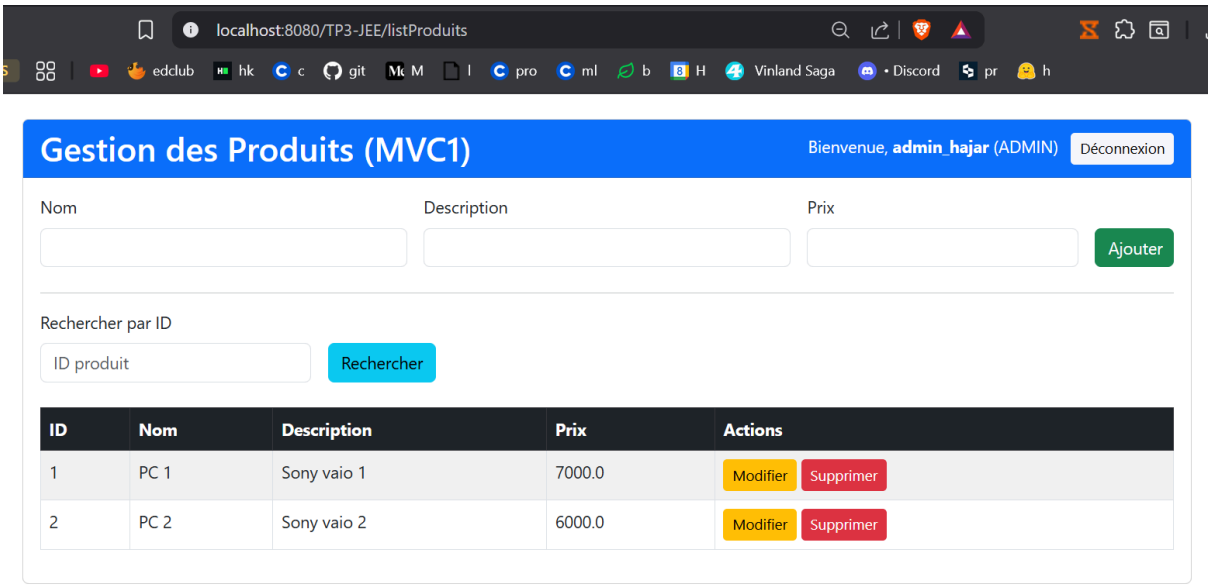


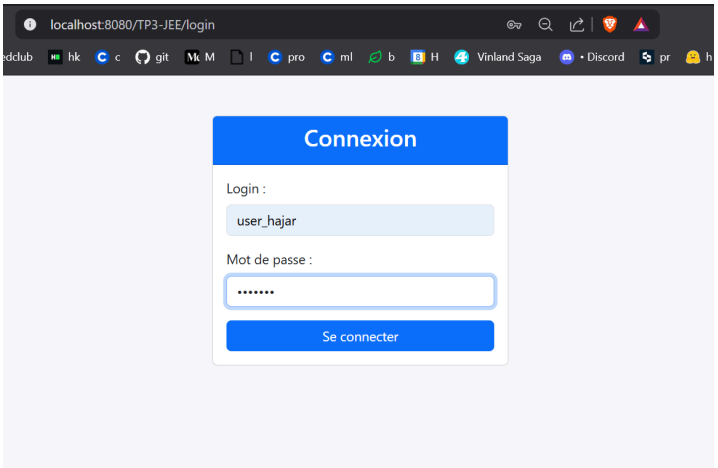


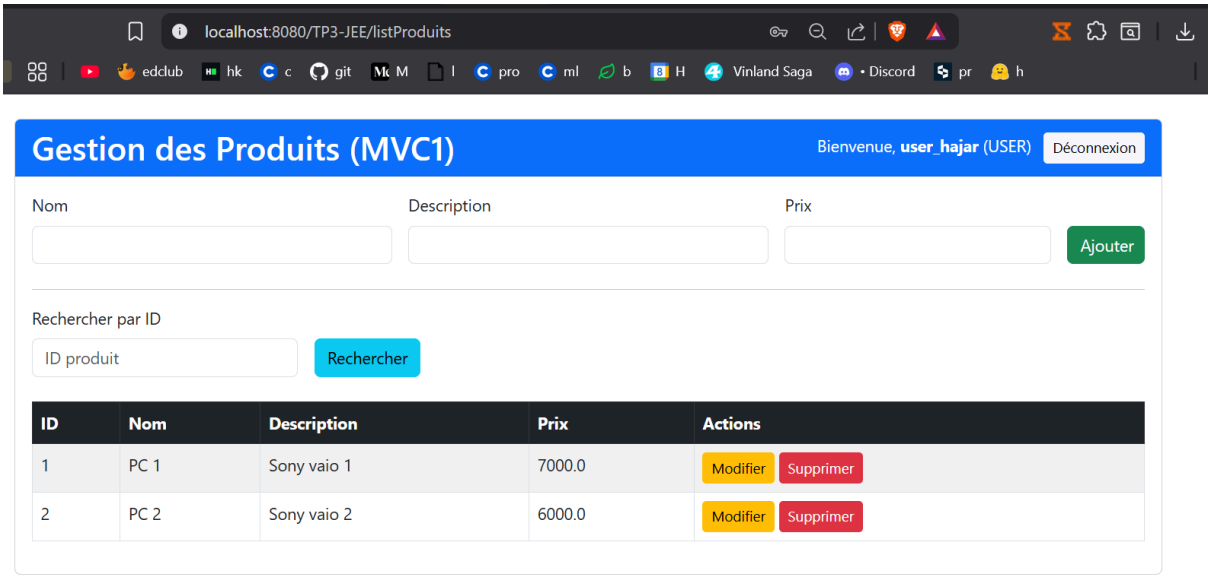
Page 3 : Modification d'un produit (ADMIN)

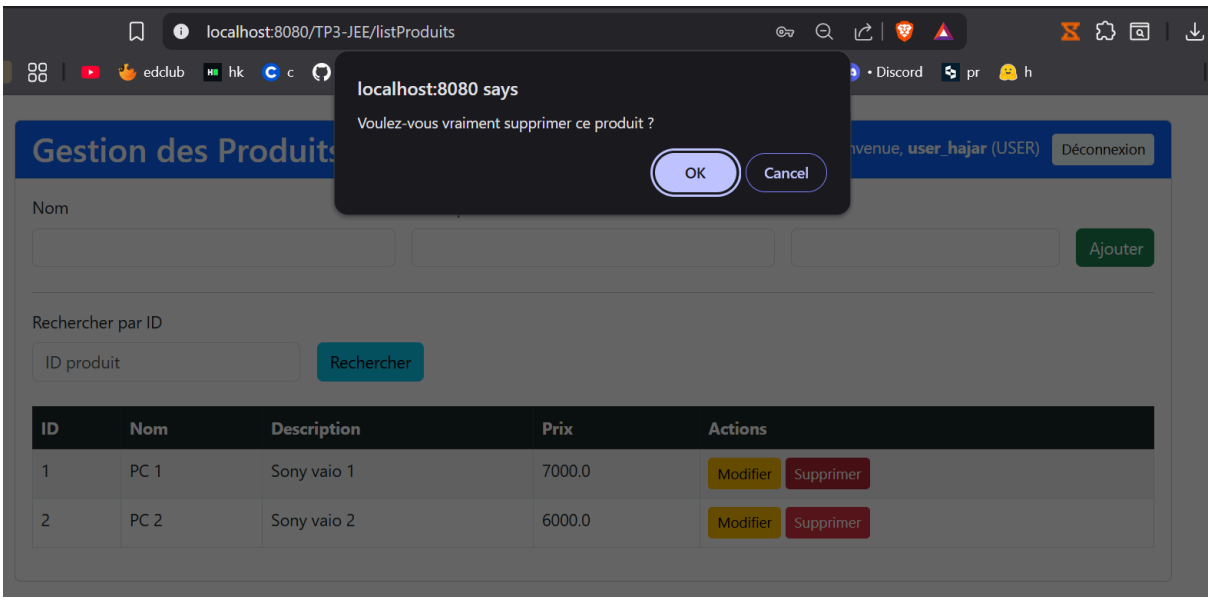


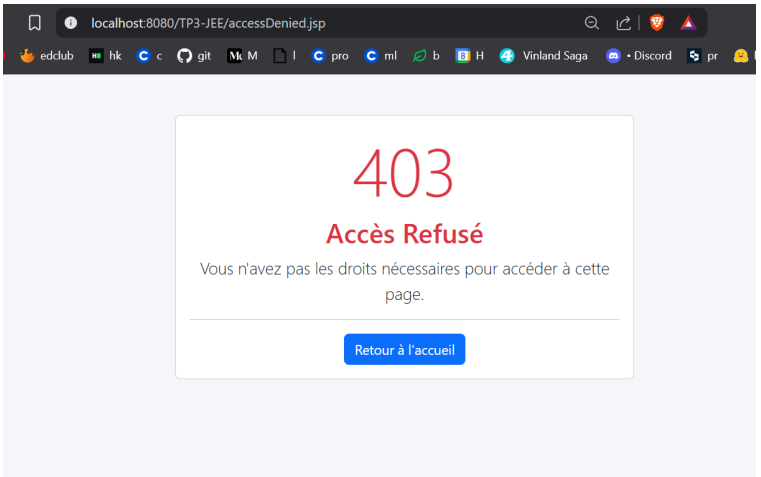


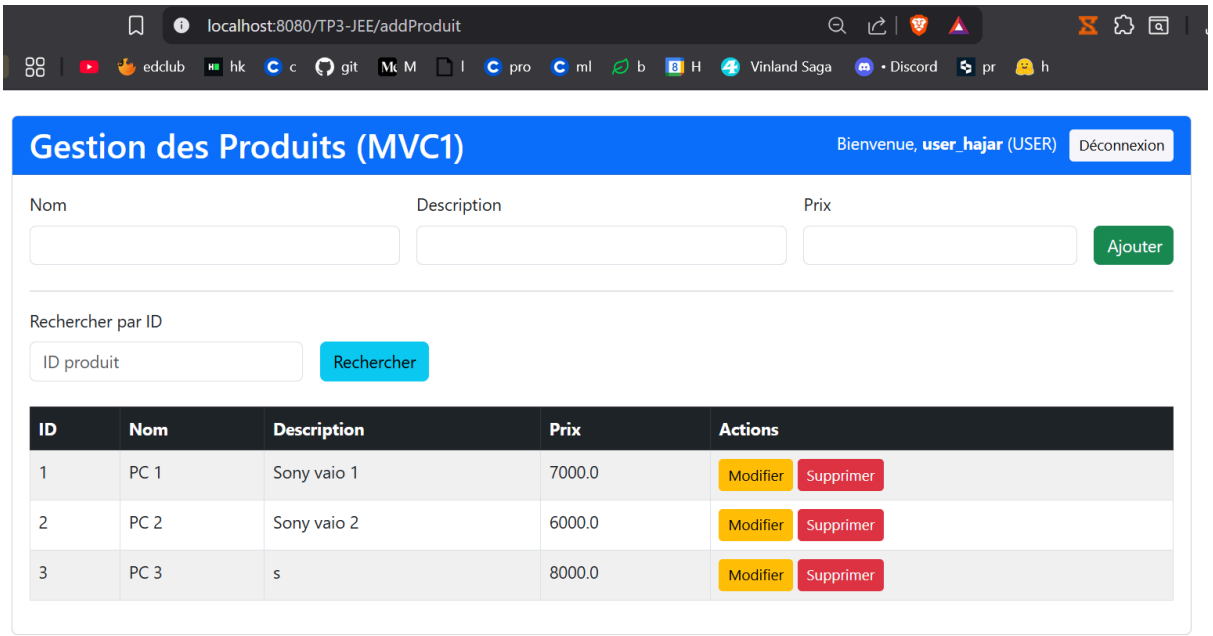
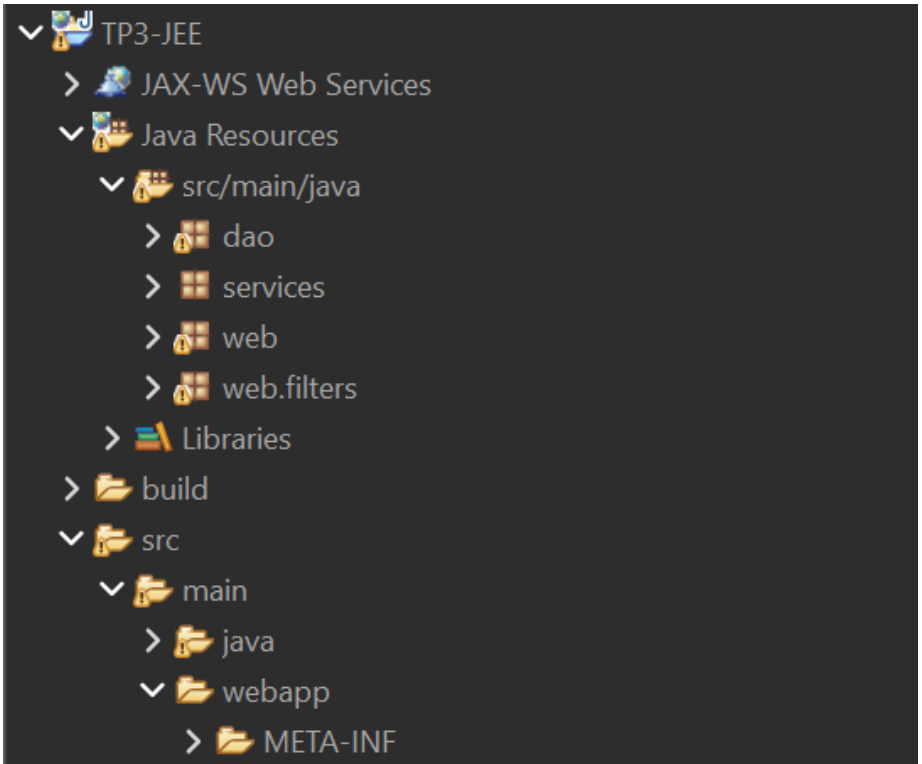



localhost:8080/TP3-JEE/login
Connexion
Login :
user_hajar
Mot de passe :
•••••••
Se connecter


localhost:8080/TP3-JEE/listProduits
Gestion des Produits (MVC1)
Bienvenue, user_hajar (USER)
Déconnexion
Nom
Description
Prix
Ajouter
Rechercher par ID
ID produit
Rechercher
ID
Nom
Description
Prix
Actions
1
PC 1
Sony vaio 1
7000.0
Modifier
Supprimer
2
PC 2
Sony vaio 2
6000.0
Modifier
Supprimer


localhost:8080/TP3-JEE/listProduits
localhost:8080 says
Voulez-vous vraiment supprimer ce produit ?
OK
Cancel
Gestion des Produits
Déconnexion
Nom
Ajouter
Rechercher par ID
ID produit
Rechercher
ID
Nom
Description
Prix
Actions
1
PC 1
Sony vaio 1
7000.0
Modifier
Supprimer
2
PC 2
Sony vaio 2
6000.0
Modifier
Supprimer





5. STRUCTURE DU PROJET